BEGIN PROMPT

Build Primer from the Paper

You have been given the complete paper

“Primer: An AI Framework for Instructor-Centered Educational Simulation Creation”

You may also have access to the Primer repository. First inspect the capabilities of the execution environment. Then read the complete paper, including its appendices, and establish the Primer Contract before choosing the implementation approach.

Your Mission

Build Primer itself.

Build a complete implementation of Primer: the reusable teacher-centered system described by the paper.

This may be an extension of the reference repository, a compatible redesign, a selectively reused implementation, or a clean alternative. It need not reproduce the reference repository’s architecture exactly. It includes:

- An educator-facing authoring system.
- A playable learner runtime.
- The artifacts, evidence, approval gates, feedback processes, and testing infrastructure connecting them.

Do not substitute any of the following for Primer:

- A summary of the paper.
- A requirements document or architecture alone.
- A static interface mock-up.
- A single simulation.
- A hard-coded demonstration.
- One end-to-end vertical slice presented as the finished product.

A vertical slice is an implementation milestone, not the finish line. Sample simulations are acceptance-test fixtures produced by Primer, not the product itself.

In a full coding environment, continue until the complete core system described below has been implemented and tested. If progress is blocked, follow the handoff procedure in Section 2. Primer must demonstrate that it can create materially different experiences from different educator inputs without application-code changes.

Make sensible, reversible decisions, record assumptions, and proceed.

---

1. Use the Paper, Repository (if you like), and Environment Correctly

Treat the three sources of direction differently:

- The paper is the pedagogical constitution. Preserve its commitments even if or when changing the implementation.
- The repository is a reference implementation and modifiable starting point, not an untouchable specification.
- The execution environment determines how much can be built and tested here.

Inspect any supplied repository before committing to its architecture. You may reuse, extend, refactor, wrap, or replace parts of it. You may work in a branch, worktree, fork, derived project, or clearly identified modified copy. If no suitable repository is available, you may create a local project.

---

2. Inspect the Environment and Declare the Build Mode

Before building, inspect whether you can:

- Read and write persistent files.
- Run shell commands and application code.
- Access or clone the repository.
- Install project-scoped dependencies.
- Run tests, linters, builds, and local services.
- Inspect a working application in a browser.
- Use model providers or deterministic substitutes.
- Create independent agents, threads, worktrees, or fresh contexts.
- Generate HTML and other downloadable artifacts.

Record the inspection and initial build status in the following artifacts or clearly identified equivalents. Keep BUILD_STATUS.html as the dashboard entry point; other records may be consolidated as described in Section 6:

- ENVIRONMENT_REPORT.md
- BUILD_STATUS.html
- BUILD_LOG.md

After the initial inspection, read the complete paper and establish the Primer Contract described in Section 3. Then declare one of these modes before choosing the implementation approach:

Mode A: Full agentic coding environment

Build and validate the complete Primer system described in this prompt. Environmental constraints may influence the stack and architecture, but they do not reduce the required core functional scope.

Mode B: Coding environment with limited orchestration

Implement the complete core application to the greatest extent the coding environment permits. Run tests. Create all fresh-agent test prompts and fixtures even if independent execution is unavailable.

Clearly identify any validation that remains pending. Do not call Primer fully validated until the required independent tests have actually run.

Mode C: Chat, document, or planning environment

Produce the strongest implementation package possible: architecture, schemas, prompts, static previews, code scaffolding, fixtures, test plans, and continuation instructions.

Do not claim that a functioning or completed Primer system has been built in Mode C.

In any build mode, if missing credentials, unavailable independent testers, or exhausted resources block further progress, preserve the working build or current implementation package and provide a resumable handoff. Identify the blocker, list all unmet requirements and pending validation, and state the exact next action needed to resume. Record the handoff and the location and startup instructions for the preserved work in BUILD_STATUS.html and the build record. Mark the run blocked or incomplete; do not claim completion or claim that unperformed tests passed. The required core scope remains unchanged.

---

3. Preserve the Primer Contract

Create the Primer Contract from the paper in PRIMER_CONTRACT.md or a clearly labeled section of a consolidated record. At minimum, maintain:

Educator authority

Educator goals, content, examples, misconceptions, criteria, constraints, and classroom knowledge must remain visible and editable. AI recommendations must be reviewable. A design cannot become classroom-approved without an explicit educator approval.

Synthetic teacher approval used in testing must be labeled as test approval, not real educator approval.

Meaningful student work

The learner must retrieve, interpret, explain, decide, diagnose, persuade, advise, revise, or otherwise perform the intended cognitive work. Primer must not solve the task prematurely or remove productive effort through excessive assistance.

Preparation, Practice, and Feedback

Every completed experience must connect:

- Preparation that gives the learner enough context and activates relevant prior knowledge.
- Practice that requires meaningful application in a scenario.
- Feedback and reflection grounded in what the learner actually did.

Productive friction

Use uncertainty, character goals, questions, constraints, consequences, and calibrated difficulty to make the instructional content necessary. Narrative must serve the learning goal rather than becoming decorative backstory.

Goal-bound adaptation

Adapt challenge, support, pacing, character behavior, and feedback in response to learner evidence while remaining within the educator-approved learning target.

Evidence-grounded feedback

Preserve relevant transcripts, decisions, artifacts, and events across phases. Distinguish observation from inference. Do not assess a capability the learner had no fair opportunity to demonstrate.

An educator or operator must be able to trace how teacher input shaped the design, runtime behavior, evidence, and feedback.

4. Build the Complete Product

Choose the architecture, stack, interface, provider, storage model, and number of agents that best serve the Primer Contract. You may depart from the paper's reference architecture if you think it best. Document that departure.

Educator-facing authoring

Build an authoring experience that allows an educator to:

- Provide or upload instructional materials.
- Describe the learners, learning objective, prior knowledge, misconceptions, intended student work, evidence of progress, and classroom constraints.
- Answer questions, preferably one consequential question at a time.
- Receive a teacher-readable proposed learning and simulation design.
- Inspect important assumptions, goals, scenario choices, criteria, and prompts.
- Edit the design or request a meaningful revision.
- Approve, reject, or return the design for further work.
- See whether a design is draft, revised, test-approved, educator-approved, rejected, or superseded.
- Build and open the resulting learner experience.

Teacher revisions must propagate to downstream prompts, criteria, runtime behavior, and feedback logic.

Learner runtime

Build a playable learner experience with three connected phases:

- Preparation.
- Practice.
- Feedback, reflection, and transfer.

Apply evidence-preserving adaptation across all three phases, within the educator-approved learning goal.

The learner runtime must:

- Make the purpose and learner role clear.
- Require the learner to perform the intended work.
- Respond to what the learner actually demonstrates.
- Maintain character consistency and stay within the educator-approved learning goal.
- Record consequential learner actions and adaptations.

- Provide evidence-grounded feedback.
- Make clear when the experience has ended.

Feedback should ask for learner self-assessment, identify specific strengths and growth areas, cite actual moments from the interaction, distinguish observation from inference, offer concrete next actions, and support transfer beyond the immediate scenario.

Reusable system behavior

Primer must not be hard-coded to one course, discipline, scenario, character, or assessment rubric.

An educator must be able to create a substantially different experience by supplying new instructional materials and making authoring decisions, all in natural language.

---

5. Make the Build Visible

Keep BUILD_STATUS.html current throughout the build. Its test results and decisions must come from the same underlying records used for the reports in Section 6; update the dashboard when those records change. It should be a self-contained operator-facing dashboard showing:

- The current milestone.
- What works now.
- What remains.
- A concise system map.
- Paper-derived commitments and where they appear.
- Links or paths to working interfaces and intermediate artifacts.
- Actual test results and known failures.
- Decisions and departures from the paper.
- Exact startup instructions.
- The next milestone.

All of the above should be visually arresting and easily explainable.

Create educator- and learner-facing HTML previews early, before the implementation becomes difficult to change. Keep creator-facing notes, debugging residue, and system prompts out of the final educator and learner interfaces.

6. Required Evidence and Documentation

Produce clear equivalents of the following artifacts. Overlapping documentation may be consolidated into fewer records, provided each required topic is clearly labeled and easy to find. Use the same underlying test results and decisions for the reports and BUILD_STATUS.html:

- README.md
- ENVIRONMENT_REPORT.md
- BUILD_STATUS.html
- BUILD_LOG.md
- PRIMER_CONTRACT.md
- PAPER_TRACEABILITY.md
- REPOSITORY_AUDIT.md, when a repository is used
- VERSION_DECISION.md
- SYSTEM_DESIGN.md
- TEACHER_CENTEREDNESS.md
- TEST_PLAN.md
- TEST_REPORT.md
- COLD_BUILD_REVIEW.md
- END_TO_END_VALIDATION.md

Also preserve the working application or implementation package, schemas, prompts, representative teacher-intake records, approved designs, experience packages, runtime transcripts, adaptation events, evidence records, and feedback records.

7. Test While Building

Testing is part of implementation.

Test:

- Application startup and documented commands.
- Schema and configuration validation.
- Authoring and runtime state transitions.
- Approval enforcement.
- Artifact and transcript persistence.
- Provider replacement or deterministic replay.
- Error and interrupted-session recovery.
- Accessibility basics.

- Teacher revisions changing the generated experience.
- Educator materials influencing the design.
- Preparation, practice, and feedback remaining connected.
- Adaptation staying tied to approved goals and learner evidence.
- Feedback citing actual learner behavior.
- Failure modes such as goal drift, character drift, over-helping, invented evidence, unsupported assessment.

For blocking or major failures, preserve the evidence, repair the implementation, rerun the relevant test, and record before-and-after results.

---

8. Run Clean-Context Validation

The build agent must not be the sole judge of its own work. Test participants must start without inheriting the builder's conversation and must not consult the builder's expected answers. Keep that conversation and those answers out of inherited context, shared notes, and accessible test materials. Give each participant only the information specified for its role below. When independent contexts are available, use different fresh agents for these roles:

Cold Build Auditor

Give the auditor only the repository or packaged build and normal startup instructions. Ask it to determine what the system is, start it, and create or locate an experience.

Record where it hesitates, misunderstands terminology, encounters setup failures, or sees internal implementation residue. After this black-box pass, give it the paper and Primer Contract for a separate conformance audit.

Fresh Teacher (simulated) Participant

Give a new teacher agent a realistic private teaching fixture (an MBA instructor, for instance), but not the paper, system prompts, intended architecture, or previous findings. For the acceptance-test experience selected independently under Section 9, use that tester's educator brief.

Require it to:

- Find the educator entry point.
- Provide instructional context and materials.
- Complete the authoring process naturally.
- Inspect the proposed design.
- Request at least one material revision.
- Verify that the revision changed the design.

- Exercise approval or rejection.
- Build and open the resulting experience.

Record the teacher-facing transcript, designs, revision, approval state, and final experience package.

Fresh Student (simulated) Participant

Give a different fresh agent only the normal student entry point, a realistic learner profile, and the information a real student would possess.

Require it to play the actual running experience and complete Preparation, Practice, and Feedback.

Record its transcript, learner work, adaptations, evidence, and feedback. Run multiple learner profiles, including a learner with a misconception and a learner who asks the system for the answer.

Independent Evidence Evaluator

Give a new evaluator the paper, Primer Contract, approved design, transcripts, events, evidence, and feedback. Ask it to determine whether:

- Teacher materials and revisions materially affected the experience.
- Approval was enforced.
- The learner performed the intended work.
- Adaptation remained goal-bound and evidence-based.
- Feedback was supported by actual behavior.
- Unelicited skills were not assessed.
- The interfaces were understandable to first-time users.
- The implementation's claims match its verified behavior.

Repair blocking and major findings, then rerun the affected validation with fresh participants.

---

9. Prove That Primer Is General

Use at least two materially different acceptance-test experiences from different curricular domains.

After the application has been built and is ready for acceptance testing, an independent tester must choose at least one of the educator briefs. The builder must not know that brief in advance. The tester must create the experience through the application's normal educator-facing authoring interface, without changing application code.

An MBA experience (such as negotiation, leadership, decision-making, finance, marketing, or organizational behavior) is acceptable as one fixture. Use a different subject area for the second fixture.

Across the two fixtures:

- Use different educator materials and objectives.
- Exercise both Roleplay and Teachback or other Templates you have created.
- Require different kinds of learner work.
- Use different evidence and feedback criteria.
- Generate and run both experiences without changing application code.

A successful MBA simulation alone does not establish that Primer has been built.

---

10. Definition of Done

Do not call Primer complete until all of the following are demonstrably true:

- The application starts using documented commands.
- An educator can provide instructional context and materials.
- Primer produces a reviewable teacher-facing design.
- The educator can revise, approve, reject, and supersede designs.
- Approval state is enforced.
- Primer generates and runs multiple experiences from different educator inputs without code changes.
- Learners complete connected Preparation, Practice, and Feedback phases.
- Practice requires meaningful learner work.
- Adaptation remains tied to the approved goal and learner evidence.
- Runtime artifacts persist across phases.
- Feedback refers to actual learner evidence.
- Educator decisions are traceable into runtime behavior.
- Two cross-domain acceptance fixtures pass, including the independent brief test in Section 9 and the MBA fixture if selected.
- Technical, pedagogical, teacher-centeredness, failure-mode, cold-reader, fresh-teacher, fresh-student, and independent-evaluator tests have run and their actual results are documented.
- Blocking and major findings have been repaired and retested.
- Privacy, accessibility, provider, and deployment limitations are documented honestly.
- No synthetic test is represented as real teacher, student, classroom, or efficacy validation.

Working Style

Proceed autonomously on reversible decisions. Do not ask the operator preference questions or wait for approval between ordinary build stages. If further progress is blocked, use the handoff procedure in Section 2.

Keep operator updates concise and concrete:

- What was created.
- What now works.
- What failed.
- What changed.
- What will happen next.

Begin Now

Inspect the environment's capabilities first and record the findings. Then read the complete paper, establish the Primer Contract, and audit any available repository. Declare the build mode, choose the implementation approach and workspace strategy, and begin building the complete Primer system. Maintain BUILD_STATUS.html and the records described in Section 6. If progress is blocked, preserve a resumable handoff as specified in Section 2.

```
END PROMPT
```